Introduction To Development With Azure Functions

Rob Windsor
rob@robwindsor.com
@robwindsor

About Me



Rob Windsor

.NET/Microsoft 365 developer, trainer, author
Microsoft MVP | Microsoft 365 Development

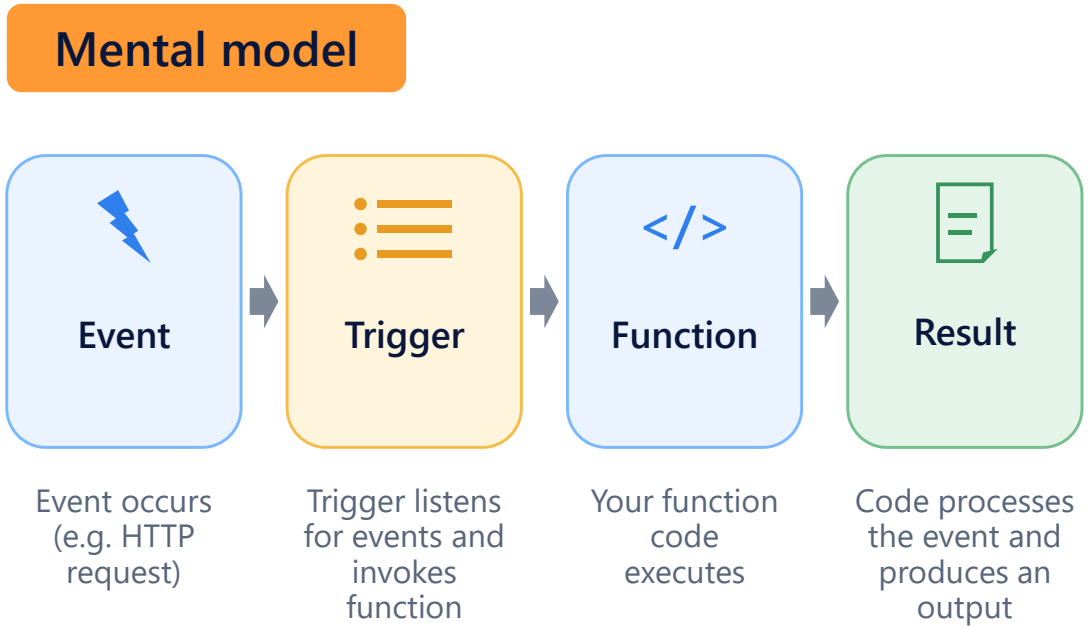


Twitter: https://twitter.com/robwindsor
LinkedIn: https://www.linkedin.com/in/rwindsor
Blog: https://robwindsor.hashnode.dev
YouTube: https://www.youtube.com/@RobWindsor
GitHub: https://github.com/rob-windsor

What is Azure Functions?

A serverless compute platform for building event-driven applications

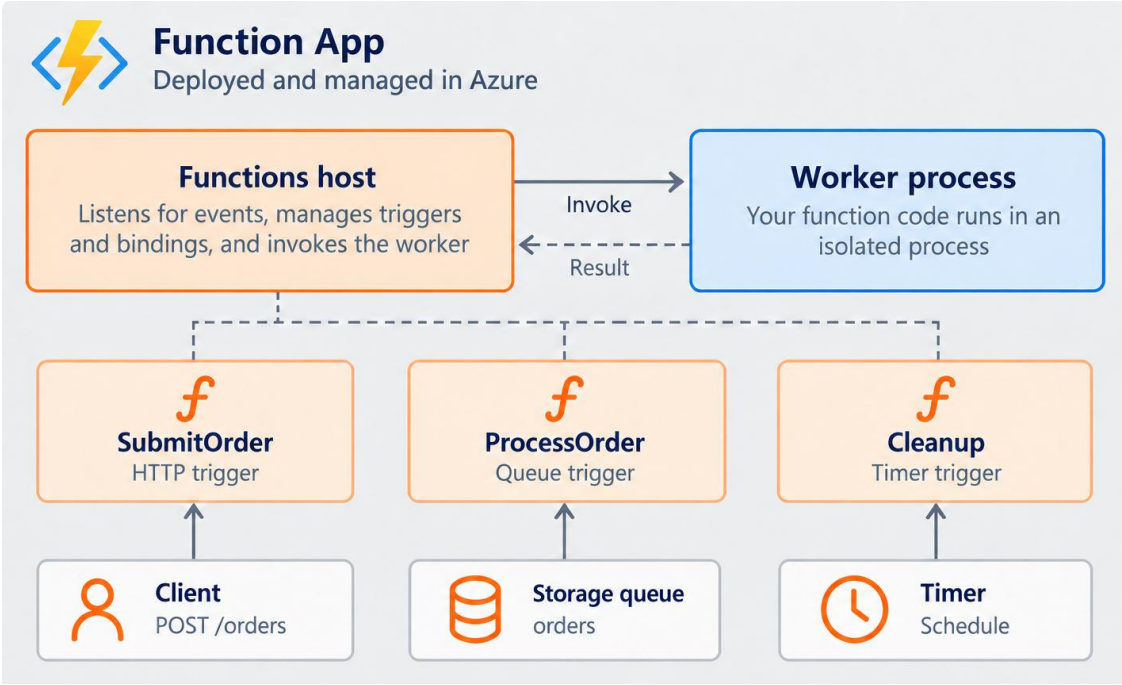
- **Run code in response to events**
  Execute functions based on HTTP requests, queue messages, timers, and other events.
- **Focus on your code**
  Azure manages the underlying infrastructure and scaling.
- **Consumption-based billing is available**
  Pay for actual execution time and resources used on supported hosting plans.
- **Support for multiple languages**
  Use C#, JavaScript/TypeScript, Python, Java, and more.
- **Not every workload is a good fit**
  Well suited to event-driven tasks and background processing. Stateful orchestration and continuously running services may require additional patterns or different hosting options.



Function Apps and Functions

A Function App is a container for one or more functions

- **Function App**
  A logical container for functions, configuration, and runtime settings.
- **Function**
  Focused code that runs when a specific event occurs.
- **Function host**
  Runtime that listens for events, manages trigger bindings, and invokes functions in the worker process.
- **Worker process**
  Your functions execute in an isolated process, separating the execution from the function host.



Triggers and Bindings

Triggers start function execution. Bindings simplify how functions read from and write to other services.

- **Triggers**
  Define what event starts your function. Each function must have exactly one trigger.
- **Bindings (Optional)**
  Declarative connections to other services, such as Azure Storage. You can also use service SDKs directly.
- **Input and output bindings**
  Input bindings provide data to your function. Output bindings write data on your function's behalf.
- **Multiple bindings**
  A function can have multiple input and output bindings.

Order Processing Scenario

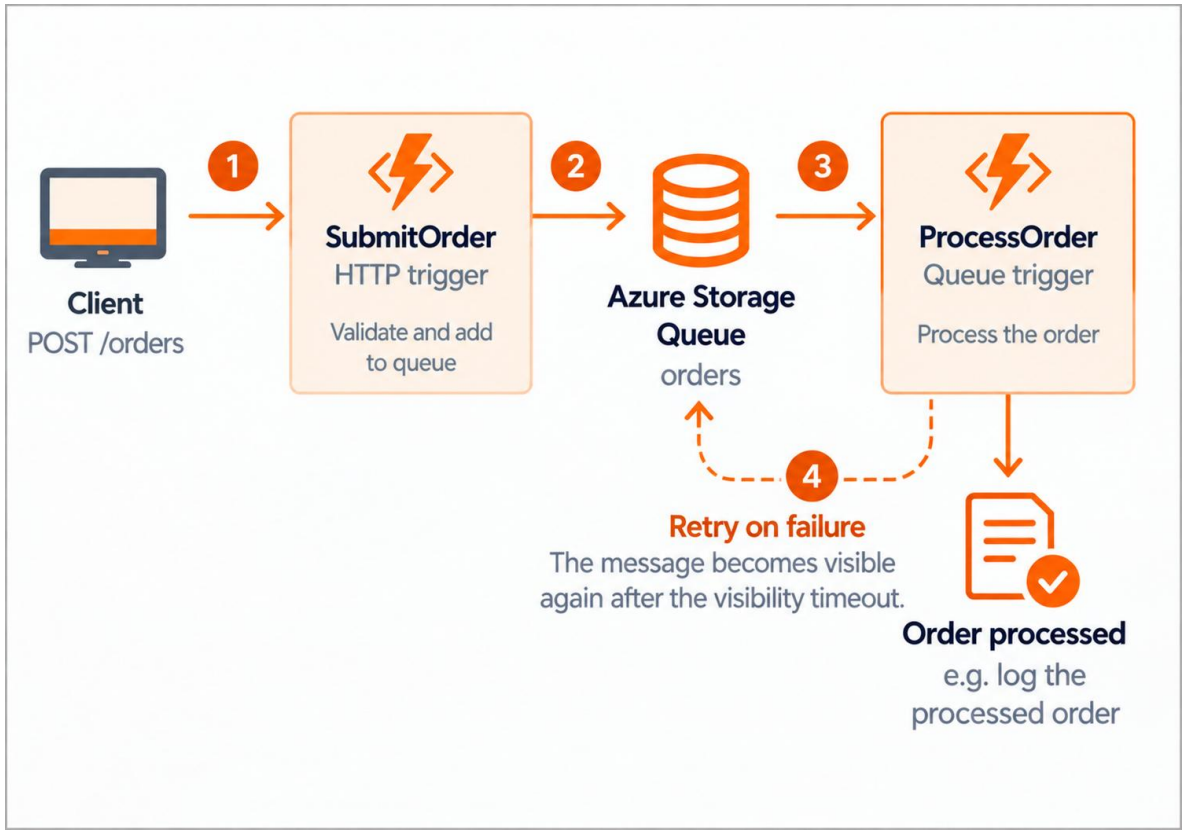
A simple example to illustrate Azure Functions: a client application submits an order. We use Azure Functions to process it asynchronously.

1. **Submit order**
   Client sends an HTTP request with order details.

2. **Add to queue**
   HTTP-triggered function validates the request and adds the order to an Azure Storage queue. Returns 202 Accepted - the order is queued, not yet processed.

3. **Process order**
   Queue-triggered function reads the message and processes the order.

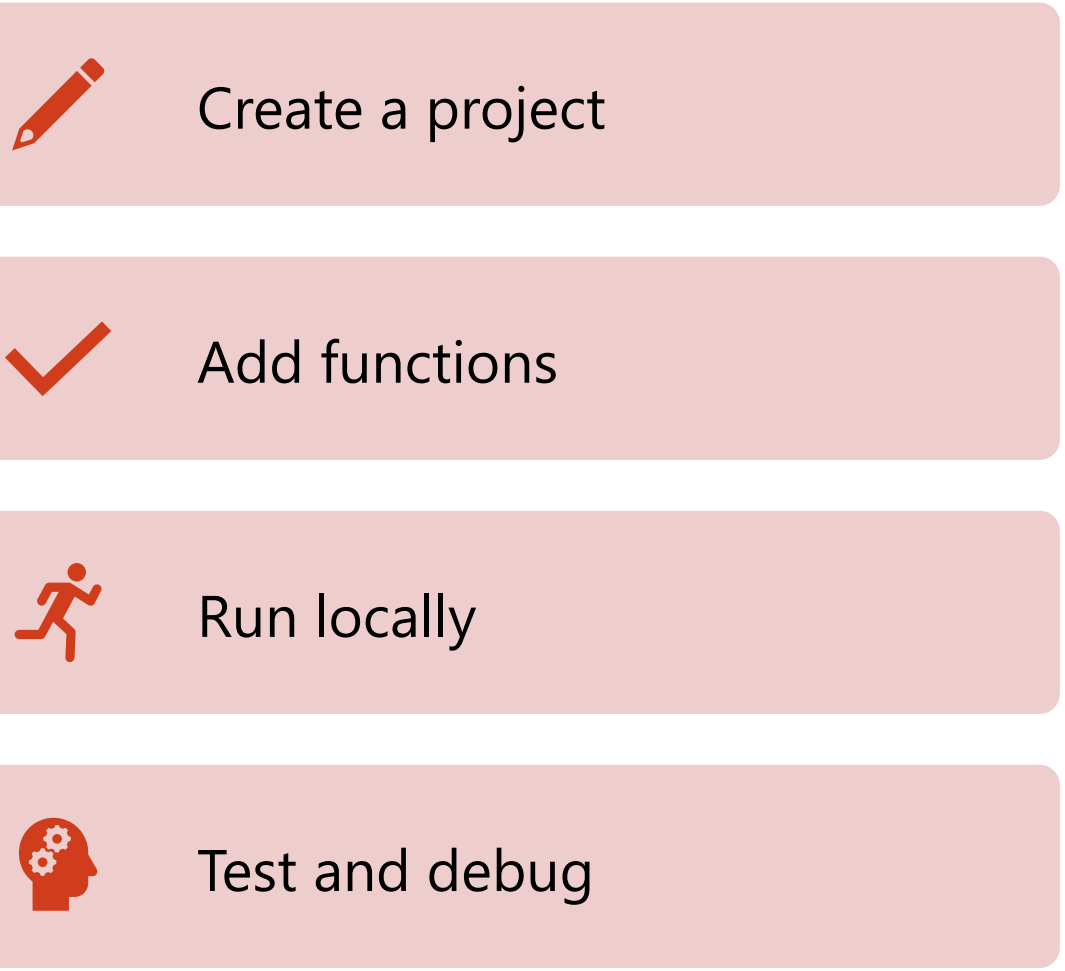
4. **Handle failures**
   If processing fails, the message is retried automatically.



Local Development

Build and test Azure Functions locally using Visual Studio or Visual Studio Code

- **Azure Functions Core Tools and Azurite**
  Runs the Functions host on your local machine. Provides templates and a local runtime. Azurite emulates Azure Storage services for local development and testing.
- **Visual Studio or Visual Studio Code**
  Create a project, add functions, and configure triggers and bindings.
- **Test locally**
  Run the Functions host and invoke your functions using HTTP requests, queue messages, or other events.
- **Local settings**
  Store configuration and connection strings in local settings or environment files.



Hosting Options

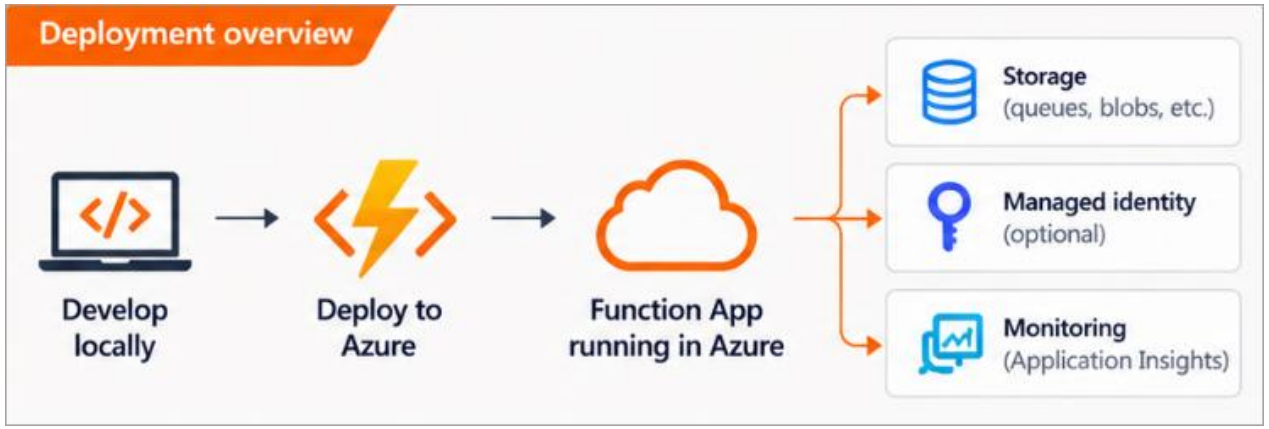
Start with Flex Consumption for new serverless applications; consider alternatives as your requirements evolve.

Hosting plan	Best for	Scaling behavior	Billing model
Flex Consumption	Event-driven applications	• Automatic scaling • Scales to zero	Consumption-based (always-ready instances billed separately)
Consumption (legacy)	Event-driven applications	• Automatic scaling • Scales to zero	Consumption-based (pay per execution)
Premium	Applications that need pre-warmed instances or consistent performance	• Automatic scaling • Minimum number of instances (pre-warmed)	Instance-based (billed for allocated compute)
Dedicated (App Service)	Long-running workloads or existing App Service environments	• Manual or automatic scaling • No scale to zero	Instance-based (billed for allocated compute)
Container Apps	Container and microservice architectures	• Automatic scaling • Can scale to zero	Consumption-based (vCPU and memory) or dedicated

Deploying to Azure

Publish your function app and run it in the cloud

- **Create Azure resources**
  Create a Function App and any other required resources (e.g. Azure Storage Account).
- **Configure settings**
  Set application settings, connection strings, and managed identities as needed.
- **Build and deploy**
  Publish your functions from Azure Functions Core Tools, the Azure CLI, Visual Studio, or Visual Studio Code.
- **Test in Azure**
  Send a request to the deployed endpoint and verify both functions executed using Application Insights.



Before You Ship

Key considerations for running Azure Functions in production.



Security

- Managed identities and secrets
- Function keys vs. authentication



Reliability

- Retries and poison queues
- Idempotent processing



Scaling and Cost

- Hosting plans and cold starts
- Execution duration and billing



Observability

- Structured logging
- Correlation and Application Insights

Same Azure Functions – Different Languages

Write functions in the language that works best for your team. Azure Functions supports multiple languages with consistent trigger and binding concepts, even though the programming APIs differ.

```
// C#

using Microsoft.Azure.Functions.Worker;
using Microsoft.Extensions.Logging;

public class ProcessOrder {
  [Function("ProcessOrder")]
  public void Run(
    [QueueTrigger("orders", Connection = "StorageConnection")]
    string message,
    FunctionContext context) {
      var logger = context.GetLogger("ProcessOrder");
      logger.LogInformation("Processing order: {Message}", message);

      // Add processing logic here
    }
}
```

```
// TypeScript

import { app, InvocationContext } from '@azure/functions';

app.storageQueue('processOrder', {
  queueName: 'orders',
  connection: 'StorageConnection',
  handler: async (message: unknown,
    context: InvocationContext) => {
    context.log('Processing order:', message);

    // Add processing logic here
  }
});
```

Key Takeaways

What you saw today applies directly to your next Function App.

Development

- Use a template to get started quickly.
- Add triggers and bindings to connect to Azure services.
- Keep your function code focused and easy to maintain.

Local Testing

- Run the Functions host locally during development.
- Use tools like the Azure Functions Core Tools.
- Test with real events and data whenever possible.

Debugging

- Use breakpoints and logging to diagnose issues.
- Check local logs and error messages for clues.
- Verify bindings and configuration when troubleshooting.

Production Readiness

- Use managed identities and secure key management.
- Plan for scaling, reliability, and cost.
- Add monitoring and logging to understand how your function runs.

Language Support

- Write functions in the language that fits your team and scenario.
- The core concepts are the same across languages.
- Choose the language that gives you the best developer experience.

Azure Services

- Functions work well with other Azure services like Storage, Service Bus, and Event Hubs.
- Use bindings to reduce boilerplate and simplify integration.
- Take advantage of the broader Azure ecosystem as your solution grows.

Other Considerations

Additional capabilities and integration options worth exploring.

Durable Functions

- Stateful, multi-step workflows
- Orchestration, checkpoints, and recovery

Authentication & Authorization

- Secure HTTP endpoints with Entra ID
- Control access using identities and roles

Model Context Protocol

- Expose functions as tools for AI agents
- Use the Azure Functions MCP extension

API Management

- Publish functions through an API gateway
- Apply policies, throttling, and routing

Resources

Azure Functions Documentation
https://learn.microsoft.com/en-us/azure/azure-functions/

Azure Functions Core Tools
https://learn.microsoft.com/en-us/azure/azure-functions/functions-run-local

Azure CLI
https://learn.microsoft.com/en-us/cli/azure/

Azure Functions .NET Isolated Worker
https://learn.microsoft.com/en-us/azure/azure-functions/dotnet-isolated-process-guide

Azure Functions with Storage Queues (trigger/bindings)
https://learn.microsoft.com/en-us/azure/azure-functions/functions-bindings-storage-queue

Azure Functions Best Practices
https://learn.microsoft.com/en-us/azure/azure-functions/functions-best-practices